

Module 12 - chapter 7

Terraform State

Q:

when I execute terraform apply, I see a message saying '*refreshing state*' what is this?

- That's Terraform checking the current real state of your AWS resources against what's in the state file — before it calculates what changes to make.

So the full flow when you run terraform apply is:

1. Refresh — Terraform calls AWS to check the current state of all resources it manages
2. Compare — compares AWS reality vs your .tf config
3. Plan — calculates what needs to change
4. Apply — makes the changes

The refresh step catches cases where someone manually changed something in AWS outside of Terraform, so it's working with accurate information before making changes.

Q:

➤ but it only looks for the resources that tf knows about, that is what you mean by "Terraform calls AWS to check the current state of all resources it manages" correct? it won't look for example for the default vpc

- Exactly — it only checks resources that are in the state file.

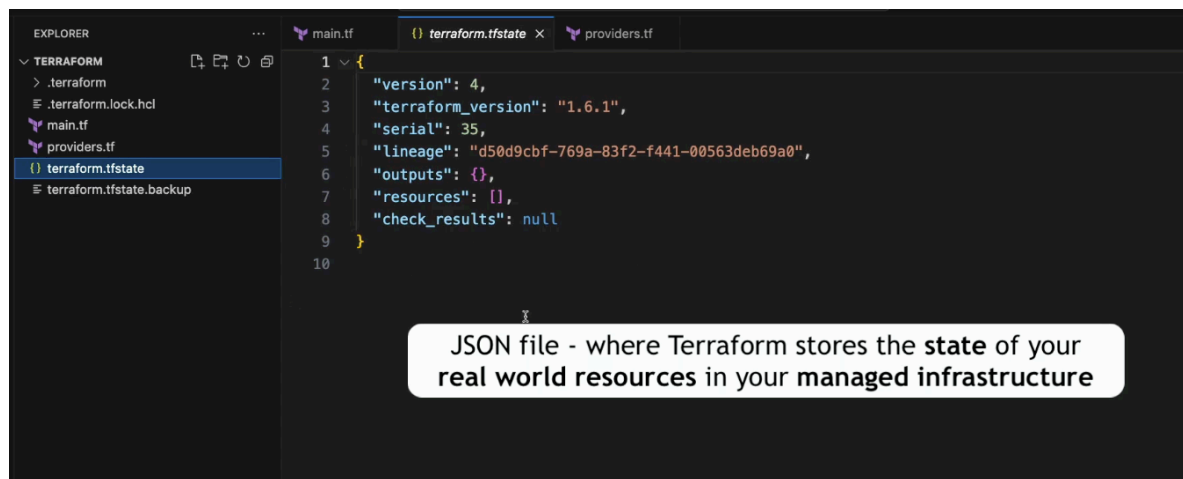
It's asking AWS "does this resource still exist and does it still look the same as I recorded?" for each resource it knows about.

The default VPC is invisible to it, just like we discussed earlier (Module 12 - chapter 5)

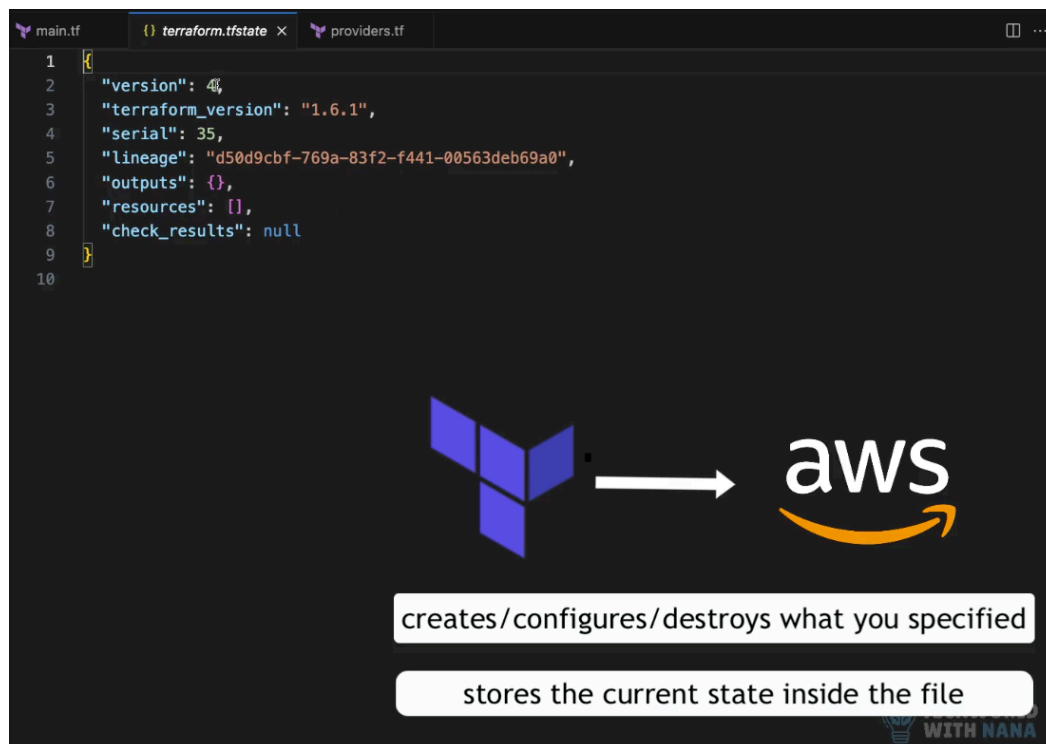
How does TF know what the current state of the resources that we defined in the tf file is?

TF created 2 files:

- terraform.tfstate.
- terraform.tfstate.backup

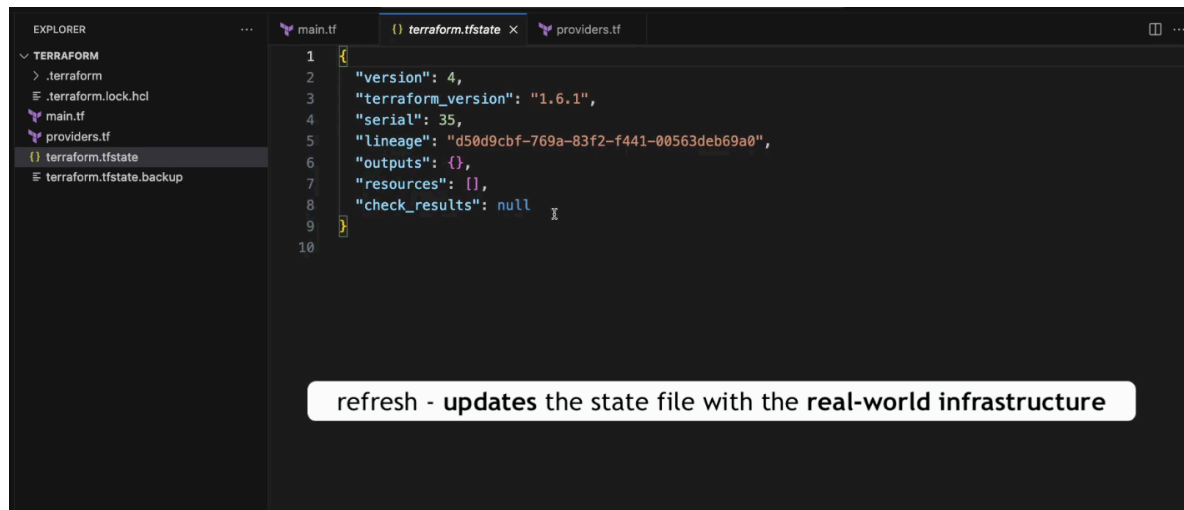


-> terraform.tfstate -> json file that gives us the resources of the "current state" at the end of executing 'terraform apply'



Everytime we apply changes this 'terraform.tfstate' gets updated -> this is the state file, a state store.

And when we execute 'terraform apply' and we see the message '**refreshing state**' the refresh is updating the state file with real-world infra.

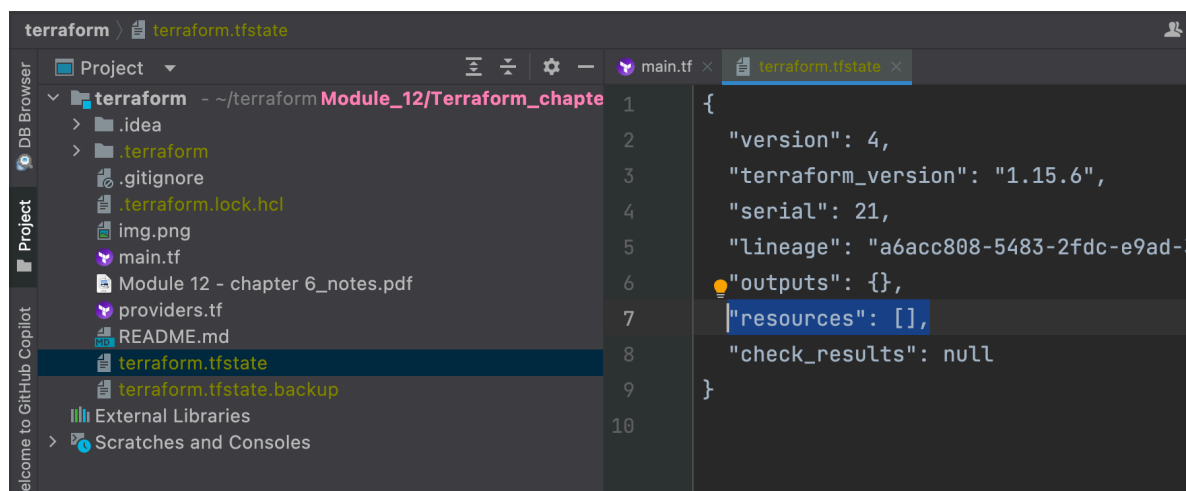


-> terraform apply

1. Refresh — calls AWS, updates state file with current reality
2. Compare — compares updated state file vs your .tf config
3. Plan — calculates changes
4. Apply — makes changes in AWS + updates state file again

So if changes are made file 'terraform.tfstate' is updated twice.

Because we deleted all the resources at the end of Module 12 - chapter 6 the list of resources is empty, see line 7:



-> 'terraform.tfstate.backup' -> is the file with the previous state

It's Terraform's safety net so you can recover the previous state if something goes wrong during apply.

```
1 {
2   "version": 4,
3   "terraform_version": "1.6.1",
4   "serial": 30,
5   "lineage": "d50d9cbf-769a-83f2-f441-00563deb69a0",
6   "outputs": {},
7   "resources": [
8     {
9       "mode": "data",
10      "type": "aws_vpc",
11      "name": "existing_vpc",
12      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
13      "instances": [
14        {
15          "schema_version": 0,
16          "attributes": {
17            "arn": "arn:aws:ec2:eu-central-1:330673547330:vpc/vpc-0371c426ad8b0bb94",
18            "cidr_block": "172.31.0.0/16",
```

Let's reapply the configuration

-> terraform apply

```
Plan: 3 to add, 0 to change, 0 to destroy.
aws_subnet.dev-subnet-2: Creating...
aws_vpc.development-vpc: Creating...
aws_subnet.dev-subnet-2: Creation complete after 1s [id=subnet-0a7eeb9709dd8682f]
aws_vpc.development-vpc: Creation complete after 1s [id=vpc-0e9064df72c28a363]
aws_subnet.dev-subnet-1: Creating...
aws_subnet.dev-subnet-1: Creation complete after 1s [id=subnet-00cd704c32a5ebc9b]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
→ terraform git:(Module_12/Terraform_chapter-7) █
```

And 'terraform.tfstate' is updated:

```
14 cidr_block = "10.0.10.0/24"
15 availability_zone = "eu-west-2a"
16 tags = {
17   Name = "subnet-1-dev"
18 }
19 }
20
21 data "aws_vpc" "existing_vpc" {
22   default = true
23 }
24
25 resource "aws_subnet" "dev-subnet-2" {
26   vpc_id = data.aws_vpc.existing_vpc.id
27   cidr_block = "172.31.48.0/20"
28   availability_zone = "eu-west-2a"
29 }

1 {
2   "version": 4,
3   "terraform_version": "1.15.6",
4   "serial": 25,
5   "lineage": "a6acc808-5483-2fdc-e9ad-34de77275cc6",
6   "outputs": {},
7   "resources": [
8     {
9       "mode": "data",
10      "type": "aws_vpc",
11      "name": "existing_vpc",
12      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
13      "instances": [
14        {
15          "schema_version": 0,
16          "attributes": {
```

We see that first it shows the data components 🙌 where we queried the default VPC

Then we see the resources components
First the subnet-1

```
7 tags = {
8   Name: "development"
9 }
10
11
12 resource "aws_subnet" "dev-subnet-1" {
13   vpc_id = aws_vpc.development-vpc.id
14   cidr_block = "10.0.10.0/24"
15   availability_zone = "eu-west-2a"
16   tags = {
17     Name: "subnet-1-dev"
18   }
19 }
20
21 data "aws_vpc" "existing_vpc" {
22   default = true
23 }
```

```
47 },
48 {
49   "mode": "managed",
50   "type": "aws_subnet",
51   "name": "dev-subnet-1",
52   "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
53   "instances": [
54     {
55       "schema_version": 1,
56       "attributes": {
57         "arn": "arn:aws:ec2:eu-west-2:706976333940:subnet/subnet-",
58         "assign_ipv6_address_on_creation": false,
59         "availability_zone": "eu-west-2a",
60         "availability_zone_id": "euw2-az2",
61         "cidr_block": "10.0.10.0/24",
62         "customer_owned_ipv4_pool": "",
```

Then subnet-2

```
20
21 data "aws_vpc" "existing_vpc" {
22   default = true
23 }
24
25 resource "aws_subnet" "dev-subnet-2" {
26   vpc_id = data.aws_vpc.existing_vpc.id
27   cidr_block = "172.31.48.0/20"
28   availability_zone = "eu-west-2a"
29   tags = {
30     Name: "subnet-2-default"
31   }
32 }
```

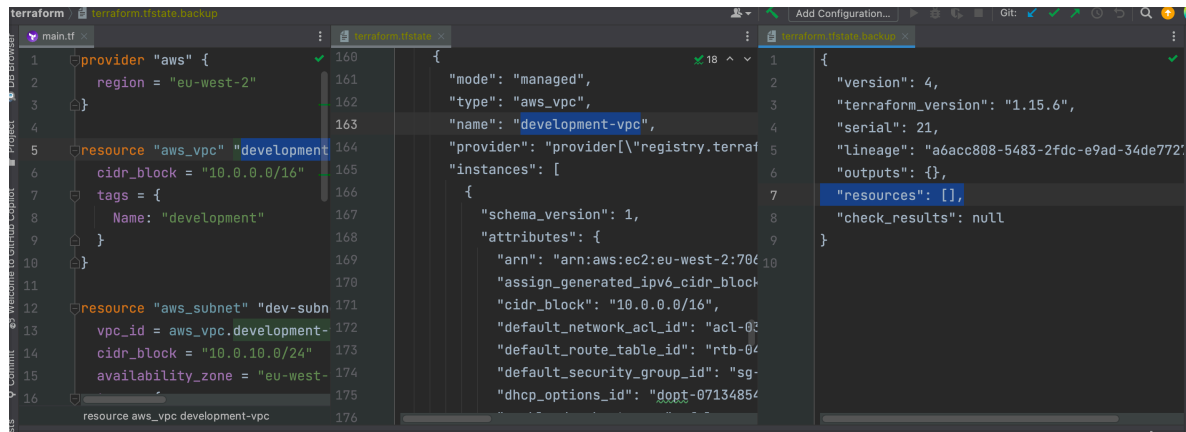
```
104 {
105   "mode": "managed",
106   "type": "aws_subnet",
107   "name": "dev-subnet-2",
108   "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
109   "instances": [
110     {
111       "schema_version": 1,
112       "attributes": {
113         "arn": "arn:aws:ec2:eu-west-2:706976333940:subnet/subnet-",
114         "assign_ipv6_address_on_creation": false,
115         "availability_zone": "eu-west-2a",
116         "availability_zone_id": "euw2-az2",
117         "cidr_block": "172.31.48.0/20",
118         "customer_owned_ipv4_pool": "",
119         "enable_dns64": false,
```

And finally the VPC that we created using TF:

```
1 provider "aws" {
2   region = "eu-west-2"
3 }
4
5 resource "aws_vpc" "development-vpc" {
6   cidr_block = "10.0.0.0/16"
7   tags = {
8     Name: "development"
9   }
10 }
11
12 resource "aws_subnet" "dev-subnet-1" {
13   vpc_id = aws_vpc.development-vpc.id
14   cidr_block = "10.0.10.0/24"
15   availability_zone = "eu-west-2a"
16 }
```

```
160 {
161   "mode": "managed",
162   "type": "aws_vpc",
163   "name": "development-vpc",
164   "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
165   "instances": [
166     {
167       "schema_version": 1,
168       "attributes": {
169         "arn": "arn:aws:ec2:eu-west-2:706976333940:vpc/vpc-0e9064",
170         "assign_generated_ipv6_cidr_block": false,
171         "cidr_block": "10.0.0.0/16",
172         "default_network_acl_id": "acl-037f67b43a776f3c1",
173         "default_route_table_id": "rtb-0430237b926948d69",
174         "default_security_group_id": "sg-09cd4b04268671a25",
175         "dhcp_options_id": "dopt-071348549901a183c",
176
```

And the backup has the previous state which had the empty resources list 📌



Reading 'terraform.tfstate' file is difficult specially if there are many resources. So we can use commands to navigate it:

-> terraform state

```
➔ terraform git:(Module_12/Terraform_chapter-7) terraform state
Usage: terraform [global options] state <subcommand> [options] [args]

This command has subcommands for advanced state management.

These subcommands can be used to slice and dice the Terraform state.
This is sometimes necessary in advanced cases. For your safety, all
state management commands that modify the state create a timestamped
backup of the state prior to making modifications.

The structure and output of the commands is specifically tailored to work
well with the common Unix utilities such as grep, awk, etc. We recommend
using those tools to perform more advanced state tasks.

Subcommands:
  identities      List the identities of resources in the state
  list            List resources in the state
  mv              Move an item in the state
  pull            Pull current state and output to stdout
  push            Update remote state from a local state file
  replace-provider Replace provider in the state
  rm              Remove instances from the state
  show            Show a resource in the state
➔ terraform git:(Module_12/Terraform_chapter-7)
```

We can see the 'state' subcommands that we can use to extract info from 'terraform.tfstate' file, for example 'list'

-> terraform state list

And we get the list of the data and resources

```
→ terraform git:(Module_12/Terraform_chapter-7) terraform state list
data.aws_vpc.existing_vpc
aws_subnet.dev-subnet-1
aws_subnet.dev-subnet-2
aws_vpc.development-vpc
→ terraform git:(Module_12/Terraform_chapter-7)
```

-> terraform state show aws_subnet.dev-subnet-1

```
→ terraform git:(Module_12/Terraform_chapter-7) terraform state show aws_subnet.dev-subnet-1
# aws_subnet.dev-subnet-1:
resource "aws_subnet" "dev-subnet-1" {
  arn                                = "arn:aws:ec2:eu-west-2:706976333940:subnet/subnet-00cd704c32a5ebc9b"
  assign_ipv6_address_on_creation   = false
  availability_zone                  = "eu-west-2a"
  availability_zone_id               = "euw2-az2"
  cidr_block                         = "10.0.10.0/24"
  customer_owned_ipv4_pool          = null
  enable_dns64                      = false
  enable_lni_at_device_index        = 0
  enable_resource_name_dns_a_record_on_launch = false
  enable_resource_name_dns_aaaa_record_on_launch = false
  id                                 = "subnet-00cd704c32a5ebc9b"
  ipv6_cidr_block                   = null
  ipv6_cidr_block_association_id    = null
  ipv6_native                       = false
  map_customer_owned_ip_on_launch   = false
  map_public_ip_on_launch           = false
  outpost_arn                      = null
  owner_id                          = "706976333940"
  private_dns_hostname_type_on_launch = "ip-name"
  region                            = "eu-west-2"
  tags                              = {
    "Name" = "subnet-1-dev"
  }
  tags_all                          = {
```

```
    "Name" = "subnet-1-dev"
  }
  vpc_id                                = "vpc-0e9064df72c28a363"
terraform git:(Module_12/Terraform_chapter-7)
```

This command is very useful as we can see all of the attributes that the resource has, as this information is coming from AWS and our tf configuration doesn't have all of the information.

Now let's remove the data wuery and the creation of the subnet in the default vpc

-> terraform apply -auto-approve

```
Plan: 0 to add, 0 to change, 1 to destroy.
aws_subnet.dev-subnet-2: Destroying... [id=subnet-0a7eeb9709dd8682f]
aws_subnet.dev-subnet-2: Destruction complete after 1s

Apply complete! Resources: 0 added, 0 changed, 1 destroyed.
→ terraform git:(Module_12/Terraform_chapter-7) × terraform state list
aws_subnet.dev-subnet-1
aws_vpc.development-vpc
→ terraform git:(Module_12/Terraform_chapter-7) ×
```

Now we only have two resources ✓

Questions

Q:

If I execute terraform apply and I don't apply the changes by confirming with 'yes' then terraform.tfstate was already updated when refreshing? will terraform.tfstate be rollback to how it was before the refresh?

A:

If you answer no at the confirmation prompt, Terraform does not roll back the state file to pre-refresh. The refresh changes stay in the state file.

However this is generally not a problem — the refresh only updates the state file to reflect what already exists in AWS. It's not creating or deleting anything, just syncing the record to reality. So the updated state file is still accurate.